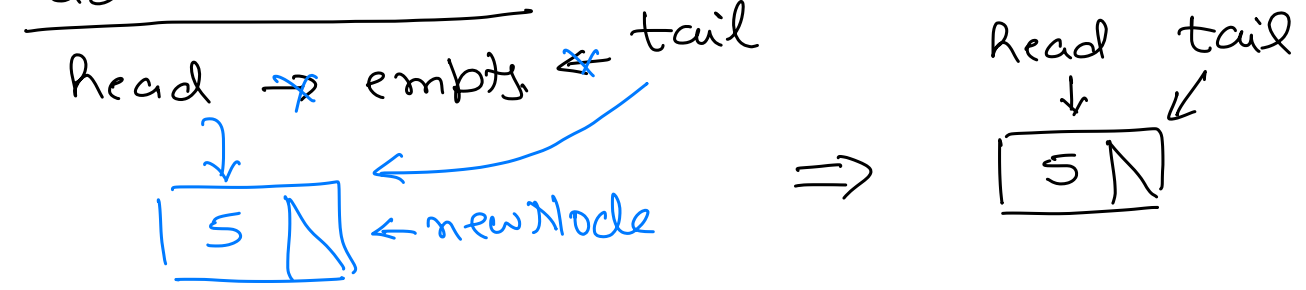
Create Linked List

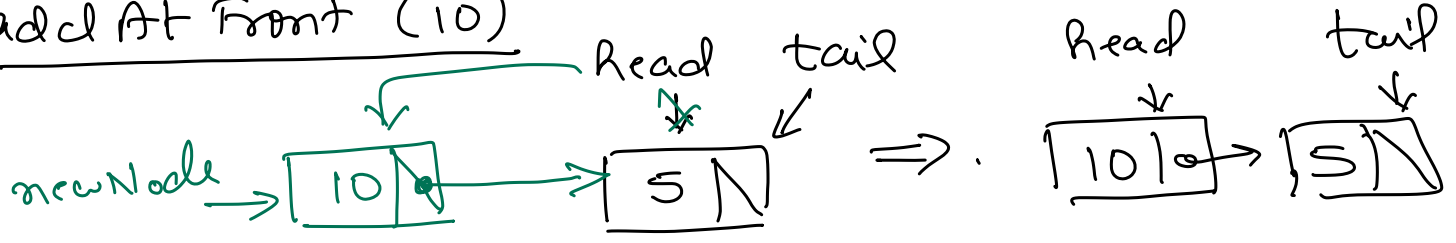
Add At Front

Initially list will be empty ⇒ new element will be the only element of the list.

add At Front (5)



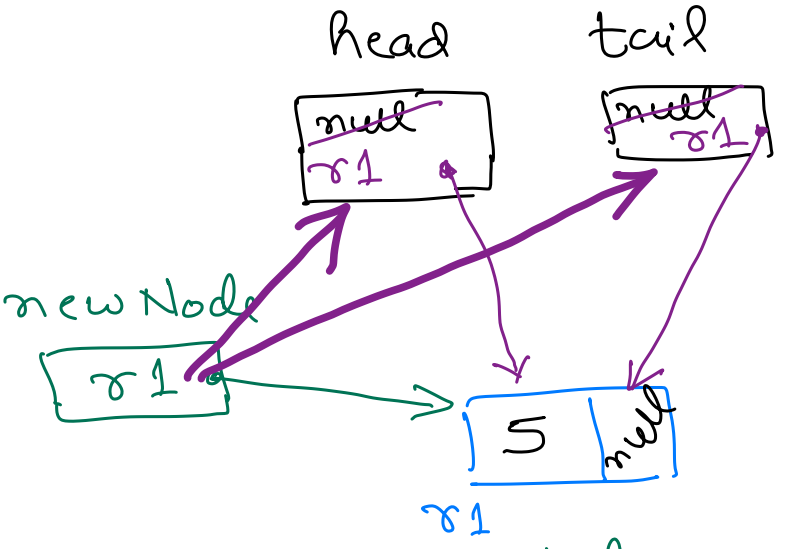
add At Front (10)



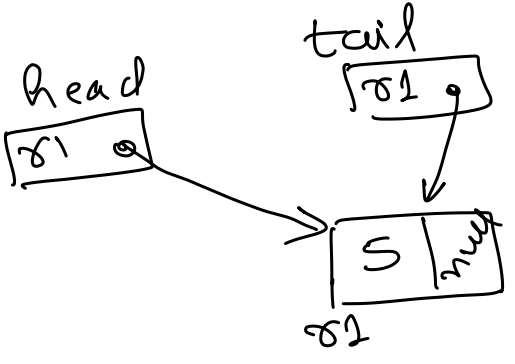
```
Class Node {
    int data;
    Node next;
}

Node head;
Node tail;
```

Reference type.



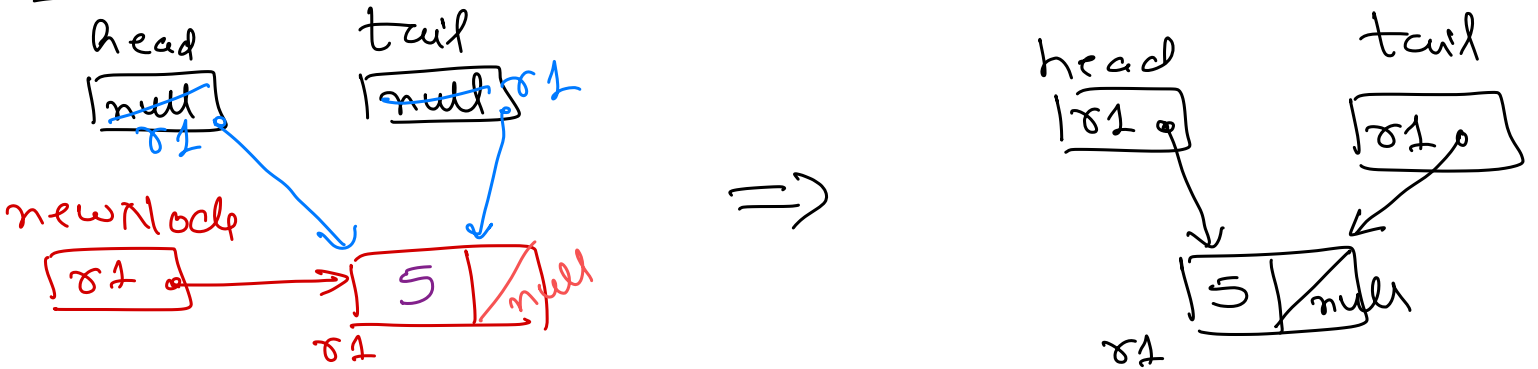

```
Node newNode = new Node();
```



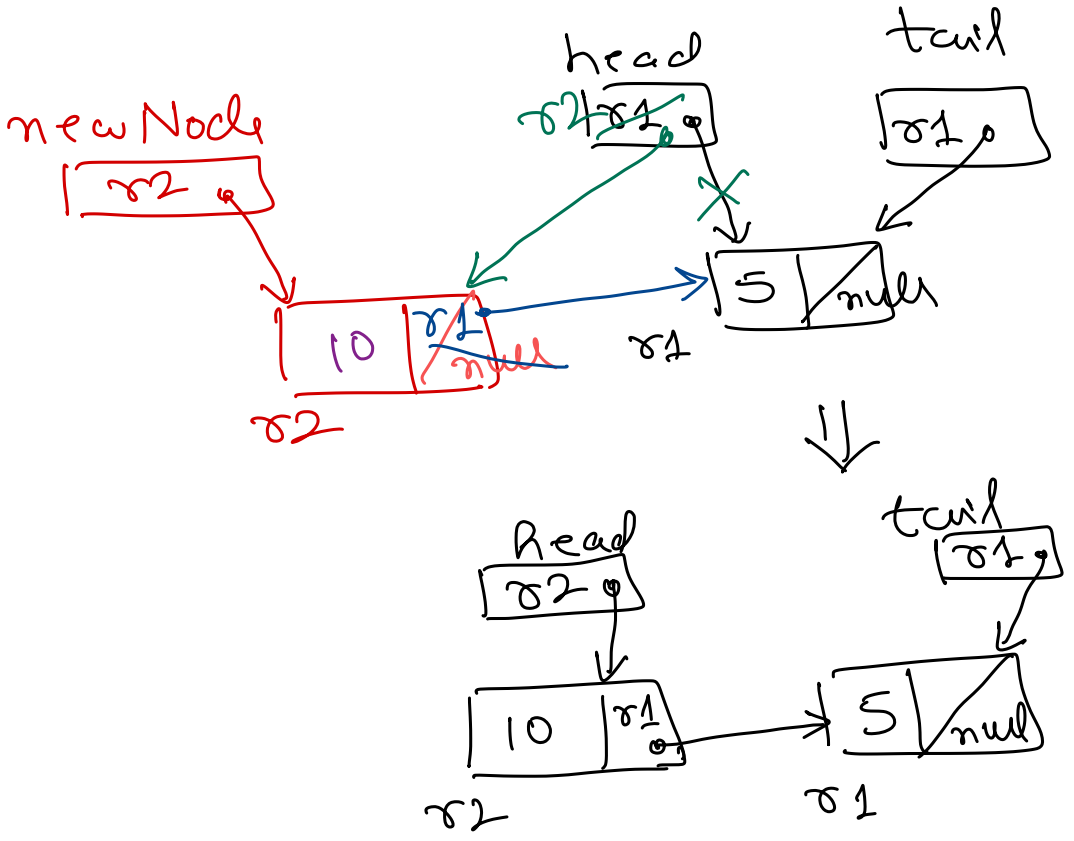
AddAtFront(element)

- Make space for new element, say newNode. → `Node newNode = new Node();`
- Store element in newNode's data. → `newNode.data = element;`
- Set newNode's next to empty. → `newNode.next = null;`
- if list is empty then → `if (head == null) {`
  - Set head and tail to newNode. → `head = newNode; tail = newNode;`
  - Stop. → `return; }`
- Set newNode's next to head. → `newNode.next = head;`
- Set head to newNode. → `head = newNode;`
- Stop.

addAtFront(5)



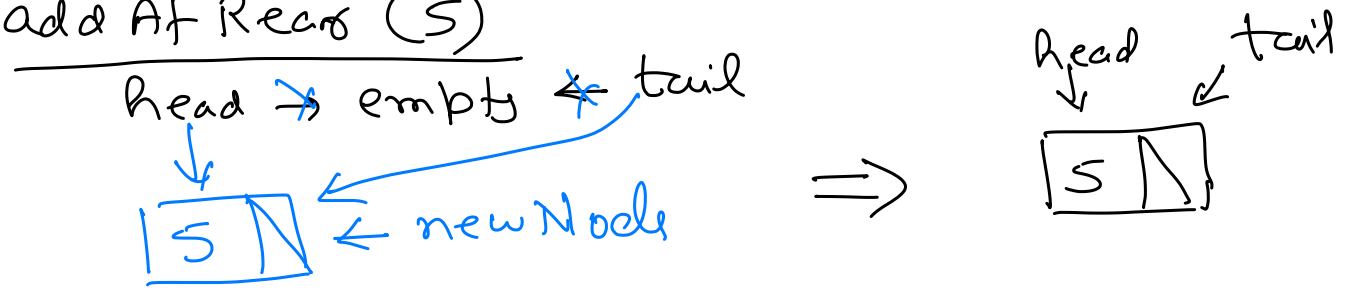
addAtFront(10)



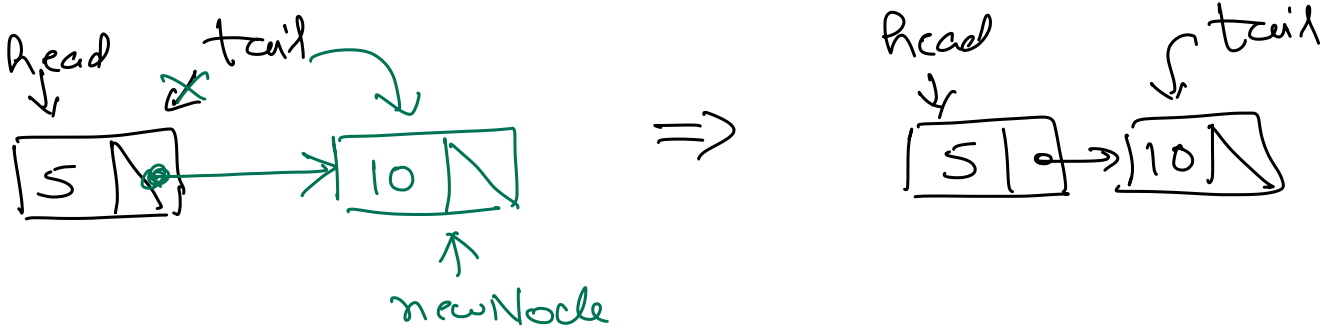
AddAtFront(element) - Optimised
- Make space for new elements, say newNode.
- Store element in newNode’s data.
- Set newNode’s next to head.
- Set head to newNode.
- if tail is empty then
  - Set tail to head.
- Stop.

addAtEnd / addAtRear

addAtRear (5)



addAtRear (10)



AddAtRear(element)

- Make space for new elements, say newNode.
- Store element in newNode's data.
- Set newNode's next to empty.
- if list is empty then
  - Set head and tail to newNode.
  - Stop.
- Set tail's next to newNode.
- Set tail to newNode.
- Stop.

```
Node newNode = new Node();
newNode.data = element;
newNode.next = null;
if (head == null) {
    head = newNode;
    tail = newNode;
    return;
}
tail.next = newNode;
tail = newNode;
```

Exercise: Implement addAtRear() without using tail (not keeping track of last node).

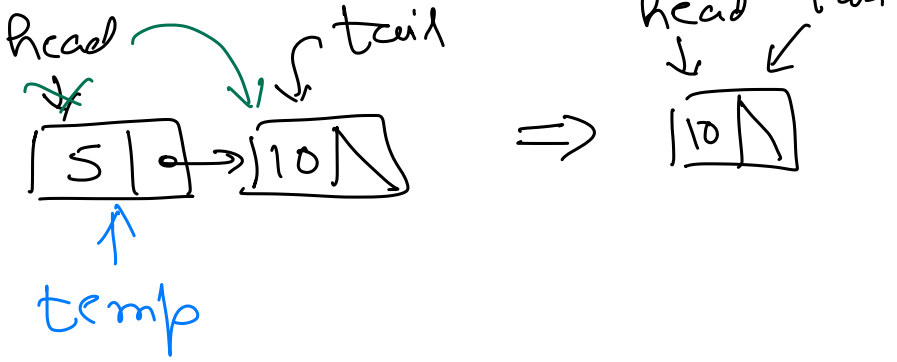
Hint: Traverse to find last node.

Delete First Node

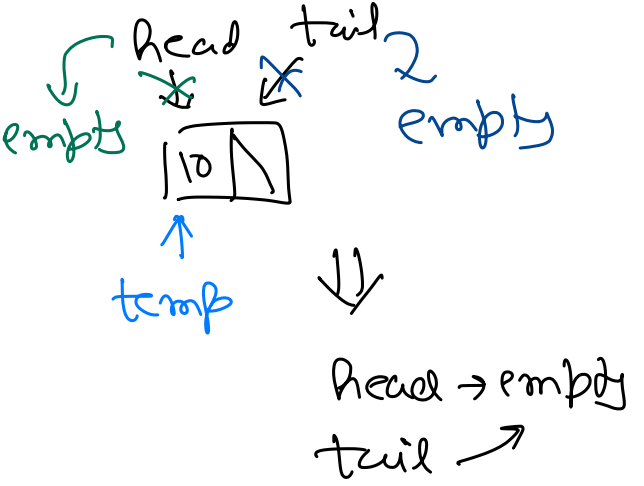
deleteFirstNode()

head → empty ← tail ⇒ Stop if list is empty.

deleteFirstNode()



deleteFirstNode()



DeleteFirstNode()

- if list is empty then
  - Stop
- Set temp to head.
- Set head to head's next.
- if list is empty then
  - Set tail to head.
- Stop.

```
if (head == null) {
    return;
}
temp = head;
head = head.next;
if (head == null) {
    tail = head;
}
```

List as ADT

```
interface List {
    void addAtFront(int element);
    void addAtRear(int element);
    int deleteFirstNode();
```

```
    void print();
};
```

Implement Stack using linked list

push()

↓

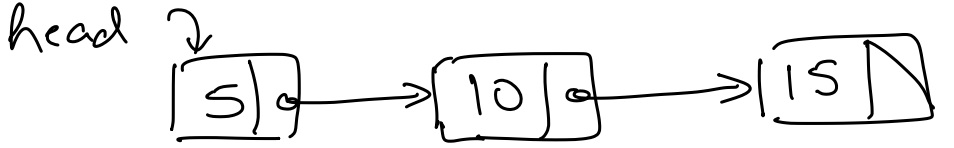
addAtFront()

pop()

↓

deleteFirstNode()

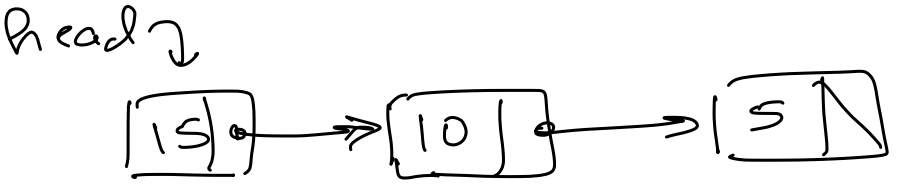
① Reverse a singly linked list.



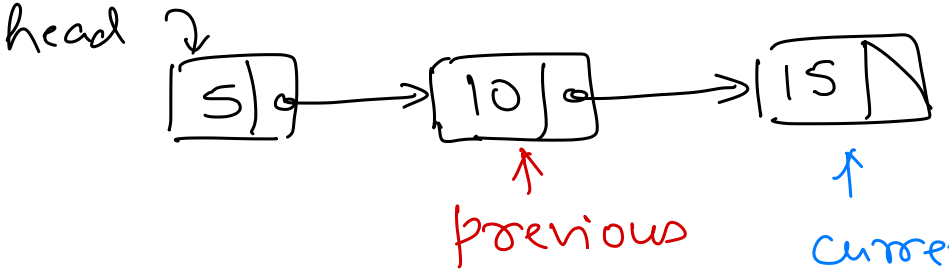
→ Use stack.

⇓ Rerese

→ Use 3 pointers



② Find 2nd last node of list.



→ Use stack

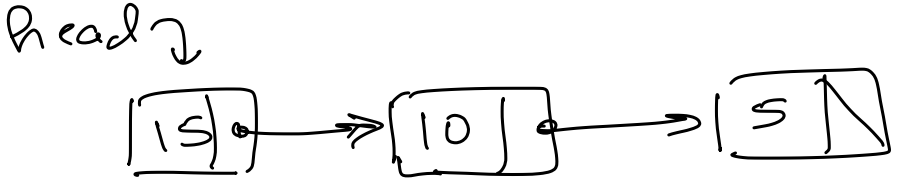
→ Use 2 pointers

③ Find k^{th} last node of list.

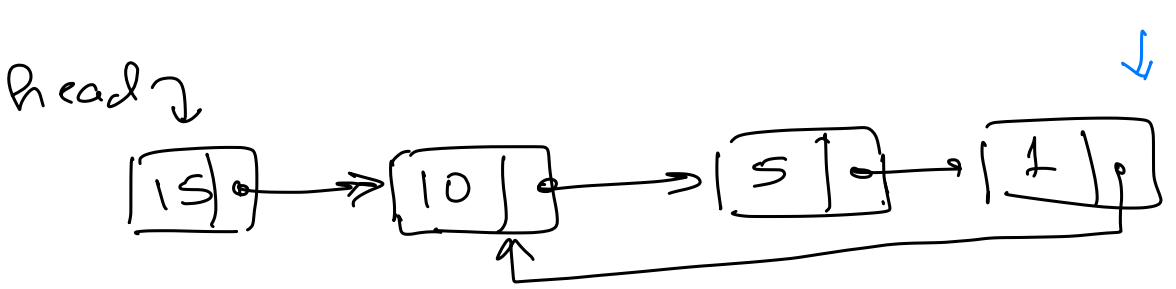
→ Use stack

→ Use 2 pointers

④ Detect if list contains a loop / cycle.



← No cycle.



→ Keep track if a node is already visited or not.

→ Use 2 pointers (hare and tortoise)

↓ fast pointer

⇓ moves ahead two nodes at a time

↓ slow pointer

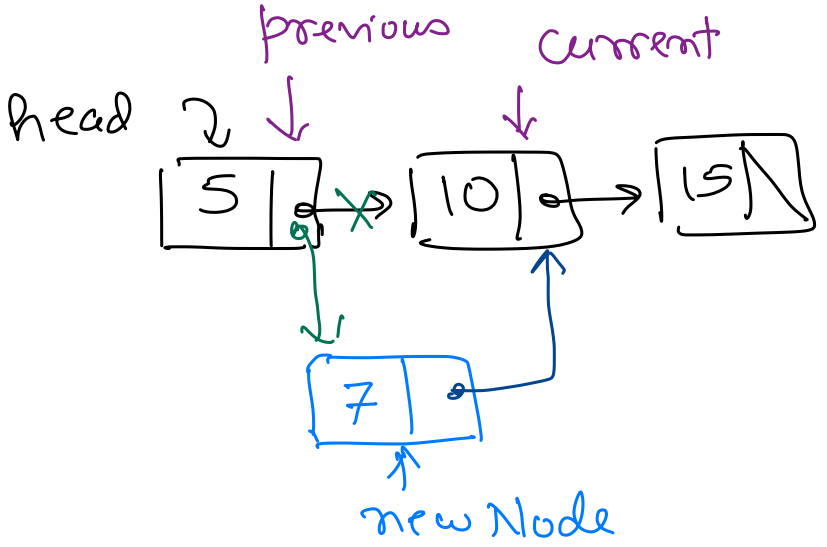
⇓ moves ahead one node at a time.

Insert in a list

→ add element at a specific position.

→ add element after a specific value.

→ add element to a sorted list.



① Create new Node

② Store element to insert in newNode's data.

③ Traverse list to find current node (node before which newNode will be added). Also have previous keep track of node before current.

→ Set current to head

→ Set previous to empty.

→ while (current is not empty)

|→ if (current.data > newNode.data)

→ End traversal.

→ Set previous to current

→ Set current to current.next

④ Add new Node between previous and current.

→ Set previous.next to newNode.

→ Set newNode.next to current

Special / Corner Cases

① Empty list.

② Adding smallest value to list. ⇒ As head needs to be changed.

③ Adding largest value to list. => If tail is used to keep track of last node.

Insert using a single pointer.

Insert(element)
- Make space for new element, say newNode.
- Store element in newNode's data.
- Set newNode's next to empty.
- if list is empty then
  - Make newNode as first (and only) node of list.
  - Stop

// List is not empty
// => Find first node having data greater than newNode's data.
- Set current to first node.
- Set previous to empty.
- while (current is not empty) do
  - if (current node's data > newNode's data) then
    // Found the node
    - End the traversal.
  - Set previous to current.
  - Move current to current's next node.

- if (previous is empty) then // newNode’s data is smallest
  // Add newNode as first node.
  - Set newNode’s next to first node.
  - Make newNode as first node.
  - Stop.

// Add newNode between previous and current
- Set newNode as next of previous.
- Set current as next of newNode.
- Stop.